

Haoxing Prep — Deep Dive on Data Structures, Algorithms, DP, and Probability

An expanded reference covering the four foundational topics with intuition, derivations, worked examples, and common pitfalls. Written at a level that assumes you're comfortable with mathematical reasoning and want to understand *why* things work, not just *what* they do.

PART 1 — Data Structures

A data structure is a contract: it promises certain operations in certain time/space. Choosing the right one is 80% of getting a problem solved in time.

1.1 Arrays

What it is: Contiguous block of memory holding elements of the same type.

Why the complexities:

- Access by index is $O(1)$ because memory is contiguous: $\text{address} = \text{base} + i \cdot \text{element_size}$.
- Insertion/deletion in the middle is $O(n)$ because all following elements must shift.
- Append is $O(1)$ amortized because Python lists double their capacity when full — over n appends, total work is $O(n)$, averaging $O(1)$ each.

Python-specific:

- Python `list` is a dynamic array, not a linked list. Random access is truly $O(1)$.
- `list.pop()` from the end is $O(1)$. `list.pop(0)` is $O(n)$. Use `deque` if you need both ends.
- Slicing `arr[i:j]` creates a new list in $O(j-i)$. Prefer indices when performance matters.

When to use:

- Whenever you need indexed access or ordered iteration.
- As the default container; almost every problem starts here.

Worked pitfall — the string concatenation trap:

```
# O(n²) — each concatenation copies the whole string
s = ""
for word in words:
    s += word

# O(n) — join collects in one pass
s = "".join(words)
```

Strings are immutable in Python, so `+=` creates a new string each time.

1.2 Hash Maps (dict) and Hash Sets (set)

What it is: A data structure that stores key-value pairs (dict) or just keys (set), with average $O(1)$ operations via a hash function mapping keys to array indices.

Why the complexities:

- The hash function distributes keys roughly uniformly. Average lookup walks through a few collisions; worst case (all keys hash to the same bucket) is $O(n)$.
- Python's dict uses open addressing with pseudo-random probing, and resizes when $\frac{2}{3}$ full.

Common patterns:

Pattern A — Frequency counting:

```
from collections import Counter
freq = Counter("abracadabra") # {'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}
```

Pattern B — Two-sum style lookup:

```
seen = {}
for i, x in enumerate(arr):
    if target - x in seen:
        return [seen[target - x], i]
    seen[x] = i
```

The insight: instead of a nested loop checking all pairs ($O(n^2)$), we query a hash map for the complement ($O(n)$ total).

Pattern C — Grouping:

```
from collections import defaultdict
groups = defaultdict(list)
for word in words:
    key = "".join(sorted(word)) # anagrams share a sorted-letter key
    groups[key].append(word)
```

When NOT to use hash maps:

- When you need sorted order — use a balanced BST or sorted list instead.
- When keys are dense integers $0..n$ — a plain array is faster and simpler.
- When you need range queries (find all keys in $[a,b]$) — use a sorted structure.

Gotcha: Only hashable (immutable) types can be keys. Lists and dicts cannot; tuples and frozensets can.

1.3 Linked Lists

What it is: A chain of nodes, each containing a value and a pointer to the next node. No random access — you traverse.

Why the complexities:

- Access: $O(n)$ — must walk from head.
- Insertion/deletion at a known node: $O(1)$ — just rewire pointers.

When they actually matter:

- LRU cache (combined with hash map for $O(1)$ access + $O(1)$ reorder).
- Implementing stacks/queues in languages without dynamic arrays.
- Interview favorites testing pointer manipulation.

Three essential patterns:

Reversing a linked list (iterative):

```
def reverse(head):
```

```

prev, curr = None, head
while curr:
    nxt = curr.next
    curr.next = prev
    prev = curr
    curr = nxt
return prev

```

Invariant: at each step, `prev` points to the reversed portion, `curr` to the unprocessed rest.

Detecting a cycle (Floyd's tortoise and hare):

```

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

```

Why it works: if there's a cycle, the fast pointer laps the slow one inside the cycle. If not, fast reaches the end.

Finding the middle:

```

def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow # when fast is at end, slow is at middle

```

1.4 Stacks

What it is: Last-In-First-Out (LIFO). Push and pop at the same end.

Python: Use a list. `stack.append(x)` to push, `stack.pop()` to pop. Both $O(1)$.

When to use — four signature patterns:

Pattern A — Matched pairs (brackets, parentheses): When you process a sequence where "openers" must match "closers" in reverse order. Push openers; on a closer, pop and

verify.

Pattern B — Nested expression evaluation: For expressions with nested operators, the stack tracks partial results at each depth.

Pattern C — DFS iteratively: Explicit stack replaces the call stack, avoiding Python recursion limits.

Pattern D — Monotonic stack (see below, this is huge): Tracks a running maximum or minimum efficiently for “next greater element” style problems.

Monotonic stack worked example — Daily Temperatures: Given `temps = [73, 74, 75, 71, 69, 72, 76, 73]`, for each day find how many days until a warmer temperature.

```
def daily_temperatures(temps):
    n = len(temps)
    ans = [0] * n
    stack = [] # stack of indices, temps at these indices are decreasing
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            ans[j] = i - j
        stack.append(i)
    return ans
```

Each index is pushed and popped at most once $\rightarrow O(n)$. The stack maintains a monotonic-decreasing sequence of temperatures; when a warmer day comes along, it “resolves” all cooler days below it.

1.5 Queues and Deques

Queue: FIFO, first in first out. Enqueue at rear, dequeue at front.

Deque: Double-ended queue, both ends support $O(1)$ push/pop.

Python: `collections.deque` — $O(1)$ on both ends. Don’t use a list for a queue;
`list.pop(0)` is $O(n)$.

Where queues shine:

BFS (Breadth-First Search): The queue naturally processes nodes in order of distance from the source, which is why BFS finds shortest paths in unweighted graphs.

Monotonic deque (sliding window maximum): Maintain a deque of indices whose values

are decreasing. The front is always the maximum of the current window.

```
from collections import deque
def max_sliding_window(nums, k):
    dq = deque() # stores indices, values at these indices are decreasing
    res = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop() # x dominates; older smaller values are useless
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft() # front is out of window
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

Each index enters and leaves the deque at most once $\rightarrow O(n)$. The elegance: by maintaining a decreasing deque, the max is always at the front.

1.6 Heaps (Priority Queues)

What it is: A complete binary tree where every parent $\leq$ its children (min-heap) or $\geq$ (max-heap). Stored compactly in an array: parent at index i , children at $2i+1$ and $2i+2$.

Why the complexities:

- Insert and extract-min: $O(\log n)$ — sift up/down along a path in a tree of height $\log n$.
- Peek min: $O(1)$ — it's at the root.
- Heapify a list: $O(n)$ — subtle result. A naive bound is $O(n \log n)$, but a careful count of work at each level gives $O(n)$.

Python: `heapq` module. Min-heap only. For max-heap, negate values on insertion.

```
import heapq
h = []
heapq.heappush(h, 3)
heapq.heappush(h, 1)
heapq.heappush(h, 4)
print(heapq.heappop(h)) # 1
```

When to use:

Top-K problems: Find the k largest/smallest elements in a stream.

- Maintain a min-heap of size k; if a new element beats the min, replace it.
- $O(n \log k)$ total, beats sorting ($O(n \log n)$) when $k \ll n$.

Dijkstra's shortest path: Always process the nearest unvisited node.

Median of a stream (two heaps): Maintain a max-heap of the lower half and a min-heap of the upper half. The median is at one of the two tops. Rebalance so sizes differ by at most 1.

Scheduling and event simulation: Always pull the next event by time.

Gotcha: Heaps give you the minimum, not sorted order. To get sorted order, repeatedly extract-min — that takes $O(n \log n)$, same as sorting.

1.7 Trees and Binary Search Trees

Tree: Connected acyclic graph. A *rooted* tree has one node designated as root; edges point away from it.

Binary tree: Each node has at most two children.

Binary Search Tree (BST): Binary tree with the property that for every node, all values in the left subtree are less, all values in the right subtree are greater.

Why BST operations are $O(\log n)$ average:

- A balanced BST has height $\log n$.
- Search, insert, delete all follow a root-to-leaf path $\rightarrow O(\log n)$.
- But a BST built by inserting sorted values becomes a linked list $\rightarrow O(n)$ per operation.
- Self-balancing BSTs (AVL, Red-Black) guarantee $O(\log n)$ by enforcing balance during updates.

Traversals (memorize these):

```
def inorder(node):    # Left, Node, Right → yields sorted order for BST
    if not node: return
    inorder(node.left)
    process(node)
    inorder(node.right)
```

```
def preorder(node):  # Node, Left, Right → useful for copying, serialization
```

```

    if not node: return
    process(node)
    preorder(node.left)
    preorder(node.right)

def postorder(node): # Left, Right, Node → useful for deletion, bottom-up DP
    if not node: return
    postorder(node.left)
    postorder(node.right)
    process(node)

```

Key insight — inorder of a BST is sorted. This powers many BST problems: “kth smallest,” “validate BST,” etc.

Level-order (BFS on tree):

```

from collections import deque
def level_order(root):
    if not root: return []
    q = deque([root])
    result = []
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result

```

1.8 Tries

What it is: A tree where each edge represents a character; paths from root to nodes represent prefixes.

Why use it: $O(L)$ insert and search where L = string length, regardless of how many strings are stored. Great for prefix queries: “how many words start with ‘pre’?”

Typical implementation:

```

class Trie:
    def __init__(self):

```



```

    self.children = {}
    self.is_end = False

def insert(self, word):
    node = self
    for ch in word:
        if ch not in node.children:
            node.children[ch] = Trie()
        node = node.children[ch]
    node.is_end = True

def search(self, word):
    node = self
    for ch in word:
        if ch not in node.children:
            return False
        node = node.children[ch]
    return node.is_end

```

When to use: Autocomplete, spell-check, IP routing, word games. Usually overkill for interview problems — hash maps often suffice.

1.9 Union-Find (Disjoint Set Union)

What it is: A data structure that tracks a partition of elements into disjoint sets, supporting two operations:

- `find(x)` : which set does `x` belong to? (returns a representative)
- `union(x, y)` : merge the two sets containing `x` and `y`.

Why the near-constant time: Two optimizations:

- **Union by rank:** always attach the shorter tree under the taller one to keep depth small.
- **Path compression:** during `find`, make every visited node point directly to the root.

With both, the amortized time is $O(\alpha(n))$, where α is the inverse Ackermann function — effectively constant ($\alpha(n) \leq 4$ for any n you'd ever see).

When to use:

- **Connected components** in a graph (can I get from A to B?).
- **Kruskal's MST:** sort edges, add each if endpoints are in different components.

- **Detecting cycles** in undirected graphs during construction.
- **Grouping problems** where “A connects to B” is an equivalence relation.

Standard template:

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # path compression
            x = self.parent[x]
        return x

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False # already connected
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True
```

1.10 Graphs: Representation Choices

Adjacency list (most common): For each node, store a list of its neighbors. Space $O(V+E)$.

```
from collections import defaultdict
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u) # undirected
```

Adjacency matrix: 2D array where $A[i][j] = 1$ if edge exists. Space $O(V^2)$. Use only for dense graphs ($E \approx V^2$) or when you need $O(1)$ edge-existence queries.

Edge list: Just a list of (u, v, weight) tuples. Useful for Bellman-Ford and Kruskal's.

Weighted graphs:

```
graph[u].append((v, weight))
```

Choose based on algorithm:

- BFS, DFS, Dijkstra: adjacency list.
 - Bellman-Ford, Kruskal's: edge list.
 - Floyd-Warshall (all-pairs shortest path): adjacency matrix.
-

PART 2 — Core Algorithms

Algorithms are recipes. The value of learning them isn't memorization — it's internalizing the *invariants* they maintain, so you can adapt them to novel problems.

2.1 Binary Search — Two Flavors

Flavor A: Find a target in a sorted array

The classic. Maintain a search interval `[lo, hi]` and halve it each step.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Invariant: if `target` exists, it's in `arr[lo..hi]` inclusive.

Complexity: $O(\log n)$. Each iteration halves the interval, so $\log_2(n)$ iterations suffice.

Flavor B: Binary search on the answer

This is the harder but more powerful pattern. You're not searching for a value in an array — you're searching for the smallest (or largest) value of *something* that satisfies a predicate.

Recognize the pattern when you see:

- "Minimize the maximum..."
- "Maximize the minimum..."
- "Find the smallest X such that [condition]..."
- "Find the largest X such that [condition]..."

Requirement: the predicate must be *monotonic*. If X satisfies the condition, so does every larger X (or every smaller X, depending on direction).

Template:

```
def smallest_feasible(lo, hi, feasible):
    # find smallest x in [lo, hi] with feasible(x) = True
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Worked example — Koko Eating Bananas (LC 875): Given piles of bananas and h hours, find the minimum eating speed k such that Koko finishes all piles in h hours.

```
def min_eating_speed(piles, h):
    def can_finish(k):
        return sum((p + k - 1) // k for p in piles) <= h
        # (p + k - 1) // k is ceil(p / k)

    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if can_finish(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Why this works: if Koko can finish at speed k, she can certainly finish at any speed > k. So "can finish in h hours" is monotonic in k. Binary search on the answer.

The mental shift: we're not searching in memory; we're searching in the space of possible

answers.

2.2 Two Pointers

Idea: Two indices traverse the array, often starting at the ends and moving toward each other, or both starting at the start but moving at different paces.

Pattern A — Sum targeting (sorted array):

Given a sorted array, find two indices whose values sum to target.

```
def two_sum_sorted(arr, target):
    i, j = 0, len(arr) - 1
    while i < j:
        s = arr[i] + arr[j]
        if s == target: return (i, j)
        elif s < target: i += 1 # need larger sum
        else: j -= 1           # need smaller sum
    return None
```

Why it works: moving *i* right increases the sum; moving *j* left decreases it. At each step, we eliminate one pointer position that can't participate in a solution.

Pattern B — Fast/slow pointers:

Already seen in linked-list cycle detection and finding the middle.

Pattern C — Partition (Dutch national flag):

Sort an array of 0s, 1s, and 2s in one pass using three pointers.

```
def sort_colors(arr):
    lo, mid, hi = 0, 0, len(arr) - 1
    while mid <= hi:
        if arr[mid] == 0:
            arr[lo], arr[mid] = arr[mid], arr[lo]
            lo += 1
            mid += 1
        elif arr[mid] == 1:
            mid += 1
        else:
            arr[mid], arr[hi] = arr[hi], arr[mid]
            hi -= 1
```

Invariant: `arr[:lo]` is all 0s, `arr[lo:mid]` is all 1s, `arr[hi+1:]` is all 2s, `arr[mid:hi+1]` is unprocessed.

2.3 Sliding Window

Idea: A window `[left, right]` slides through the array. Expand by moving `right`; contract by moving `left`. The window maintains some state (sum, frequency count, min/max).

When to use: Problems asking for the longest/shortest contiguous subarray satisfying a property.

Template:

```
def sliding_window(arr):
    left = 0
    best = 0
    state = ... # whatever you track
    for right in range(len(arr)):
        # include arr[right] in state
        while not valid(state):
            # exclude arr[left] from state
            left += 1
        best = max(best, right - left + 1)
    return best
```

Worked example — Longest Substring Without Repeating Characters (LC 3):

```
def longest_unique_substring(s):
    last_seen = {}
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

Why the complexity is $O(n)$, not $O(n^2)$: Both `left` and `right` move only forward, each at most n times. Even though the inner `while` loop looks nested, the total work across all iterations is bounded by $2n$.

Variant — fixed-size window: If the window size is fixed (e.g., “find max sum subarray of size k”), you just slide and update incrementally:

```
def max_sum_k(arr, k):
    s = sum(arr[:k])
    best = s
    for i in range(k, len(arr)):
        s += arr[i] - arr[i - k]
        best = max(best, s)
    return best
```

2.4 Breadth-First Search (BFS)

Idea: Explore a graph level by level, using a queue. Nodes at distance d from the source are processed before nodes at distance $d+1$.

Why shortest path in unweighted graphs: since BFS visits all distance- d nodes before any distance- $(d+1)$ node, the first time you reach a node is via a shortest path.

Template:

```
from collections import deque

def bfs_shortest_path(start, target, graph):
    if start == target: return 0
    visited = {start}
    q = deque([(start, 0)])
    while q:
        node, dist = q.popleft()
        for nb in graph[node]:
            if nb == target:
                return dist + 1
            if nb not in visited:
                visited.add(nb)
                q.append((nb, dist + 1))
    return -1 # not reachable
```

Variant — multi-source BFS: Instead of starting from one node, seed the queue with many. Useful for problems like “find the nearest exit” or “rotting oranges” where multiple sources spread simultaneously.

Gotcha: Don’t forget to mark a node as visited *when you add it to the queue*, not when you pop it. Otherwise you’ll add it multiple times.

2.5 Depth-First Search (DFS)

Idea: Go as deep as possible along one path before backtracking.

Recursive form:

```
def dfs(node, graph, visited):
    if node in visited: return
    visited.add(node)
    # pre-order work here
    for nb in graph[node]:
        dfs(nb, graph, visited)
    # post-order work here
```

Iterative form (with explicit stack):

```
def dfs_iter(start, graph):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited: continue
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                stack.append(nb)
```

When to prefer DFS over BFS:

- When you need to explore *all* paths (DFS naturally backtracks).
- When the graph is tree-shaped and you want to aggregate child information (post-order).
- When you have recursion depth budget and stack is simpler than queue.

Classic DFS applications:

- **Connected components:** DFS from each unvisited node; count how many times you started.
- **Cycle detection in directed graphs:** use three colors (white/gray/black). Gray-to-gray edge = cycle.
- **Topological sort:** DFS, push to stack on post-order; reverse the stack.

- **Detecting bridges and articulation points:** Tarjan's algorithm.
-

2.6 Dijkstra's Shortest Path

Problem: Given a weighted graph with *non-negative* edges and a source, find shortest distances to all nodes.

Idea: Greedily expand the frontier by repeatedly extracting the nearest unvisited node. Once we finalize a node's distance, it never changes — because any future path to it via other nodes would be longer (non-negative edges ensure this).

Why non-negative edges matter: the greedy choice relies on the fact that adding more edges can't decrease the total. Negative edges break this (use Bellman-Ford instead).

Template with priority queue:

```
import heapq

def dijkstra(start, graph, n):
    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)] # (distance, node)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue # stale entry, skip
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist
```

Complexity: $O((V+E) \log V)$ with a binary heap.

"Stale entry" trick: since Python's `heapq` doesn't support decrease-key, we just push a new entry and skip outdated ones via the `if d > dist[u]: continue` check.

2.7 Bellman-Ford

Problem: Shortest paths from a source, *allowing negative edge weights*. Also detects negative cycles.

Idea: Relax every edge $V-1$ times. After k passes, $\text{dist}[v]$ holds the shortest path using at most k edges. Since any simple path uses $\leq V-1$ edges, $V-1$ passes suffice. A V th pass that still finds improvements means there's a negative cycle.

```
def bellman_ford(edges, n, source):
    dist = [float('inf')] * n
    dist[source] = 0
    for _ in range(n - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated: break # early termination
    # Negative cycle check
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            return None # negative cycle detected
    return dist
```

Complexity: $O(V \cdot E)$.

Killer application for your role: currency arbitrage detection. Represent each currency as a node, each exchange rate as an edge with weight $-\log(\text{rate})$. Multiplying rates becomes summing logs; a negative cycle in log-space is a multiplicative cycle > 1 in rate-space — arbitrage.

2.8 Topological Sort

Problem: Order the nodes of a DAG so that for every directed edge (u, v) , u comes before v .

Two algorithms:

Kahn's algorithm (BFS on in-degrees):

```
from collections import defaultdict, deque

def topo_sort_kahn(n, edges):
    graph = defaultdict(list)
    indeg = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indeg[v] += 1
```

```

q = deque(i for i in range(n) if indeg[i] == 0)
order = []
while q:
    u = q.popleft()
    order.append(u)
    for v in graph[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
if len(order) != n:
    return None # cycle exists
return order

```

DFS with post-order stack: DFS, push to stack on post-order return. Reverse at end.

Applications: course scheduling, build systems (make, bazel), Excel formula evaluation, any dependency resolution.

2.9 Sorting

You rarely implement sorting yourself, but you should know:

Mergesort: always $O(n \log n)$, stable, uses $O(n)$ extra space. Divide array in half, sort each half, merge.

Quicksort: average $O(n \log n)$, worst $O(n^2)$ (with bad pivot), usually in-place. Pick a pivot, partition, recurse.

Heapsort: $O(n \log n)$ worst case, in-place, not stable. Build a heap, repeatedly extract min.

Python's `sort`: Timsort, a hybrid of mergesort and insertion sort. $O(n \log n)$, stable, exploits pre-sorted runs for better average performance.

When to use counting sort: integers in a bounded range $[0, k]$. Time $O(n+k)$, space $O(k)$. Beats comparison sorting's $\Omega(n \log n)$ lower bound by not being comparison-based.

Custom comparators:

```

from functools import cmp_to_key
# Sort strings so their concatenation is largest
arr.sort(key=cmp_to_key(lambda a, b: -1 if a+b > b+a else 1))

```

2.10 Recursion and Backtracking

Backtracking: exhaustive search that prunes branches that can't lead to solutions.

Template:

```
def backtrack(state, choices):
    if is_goal(state):
        record(state)
        return
    for choice in choices:
        if valid(state, choice):
            apply(state, choice)
            backtrack(state, new_choices(choices, choice))
            undo(state, choice) # backtrack
```

Classic problems: permutations, combinations, subsets, N-queens, Sudoku solver, word search.

Pruning is the art: without pruning, backtracking is exponential. With good pruning (early detection of infeasibility), it's often practical on small inputs.

PART 3 — Dynamic Programming

DP is the single most common topic in Chinese coding assessments. Master it, and you've unlocked a huge fraction of the problem space.

3.1 What makes a problem DP

Two properties must hold:

Optimal substructure: an optimal solution to the problem contains optimal solutions to subproblems. If this fails, DP doesn't work.

Overlapping subproblems: the same subproblems are solved repeatedly in a naive recursion. If every subproblem is unique, there's nothing to memoize — and DP degenerates to divide-and-conquer.

3.2 The state-recurrence workflow

Every DP problem follows the same workflow:

1. **Define the state.** What does `dp[i]` (or `dp[i][j]`, etc.) *mean*? Be precise. "The maximum profit using the first *i* items" is a state definition; "something about items" is not.
2. **Write the recurrence.** How does `dp[i]` depend on earlier states? This is where you enumerate choices: at step *i*, what options do you have?
3. **Identify base cases.** What's `dp[0]`? What are the smallest inputs?
4. **Determine iteration order.** For `dp[i]` to be computable, all its dependencies must already be computed. For a 1D array, that's usually left-to-right. For 2D, it depends on which corners the recurrence references.
5. **Extract the answer.** Is it `dp[n]`? `max(dp)`? Something else?

Pro tip: write the recurrence before you write any code. If you can't state the recurrence in English, you're not ready to code.

3.3 Top-down vs bottom-up

Top-down (memoized recursion):

```
from functools import lru_cache

@lru_cache(maxsize=None)
def solve(i, j):
    if base_case: return ...
    return ... combine solve(i-1, j) solve(i, j-1) ...
```

Pros: follows the problem's recursive structure naturally; easy to write; only computes states you actually need.

Cons: recursion overhead; risk of hitting Python's recursion limit.

Bottom-up (tabulation):

```
dp = [[0] * (n+1) for _ in range(m+1)]
for i in range(1, m+1):
    for j in range(1, n+1):
        dp[i][j] = ...
return dp[m][n]
```

Pros: no recursion overhead; easier to optimize space; iteration order is explicit.

Cons: computes all states (even unneeded ones); less intuitive.

Rule of thumb: write top-down to get it working, then convert to bottom-up if speed or space matters.

3.4 Classic patterns with derivations

Pattern — Fibonacci / Climbing Stairs

State: $dp[i]$ = number of ways to reach step i . Recurrence: $dp[i] = dp[i-1] + dp[i-2]$ (you got here from $i-1$ or $i-2$). Base: $dp[0] = 1$, $dp[1] = 1$. Space optimization: only need last two values, so $O(1)$ space.

Pattern — House Robber

State: $dp[i]$ = max money robbing from houses $0..i$. Recurrence: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$.

- Skip house i : inherit $dp[i-1]$.
- Rob house i : add $\text{nums}[i]$ to $dp[i-2]$ (can't rob $i-1$).

Pattern — Maximum Subarray (Kadane's)

State: $dp[i]$ = max sum of a subarray ending exactly at i . Recurrence: $dp[i] = \max(\text{nums}[i], dp[i-1] + \text{nums}[i])$.

- Start fresh at i , or extend the previous subarray. Answer: $\max(dp)$.

Pattern — Longest Increasing Subsequence (LIS)

State: $dp[i]$ = length of LIS ending at i . Recurrence: $dp[i] = 1 + \max(dp[j] \text{ for } j < i \text{ if } \text{nums}[j] < \text{nums}[i])$. $O(n^2)$ straightforward, but there's an $O(n \log n)$ variant using patience sorting + binary search.

Pattern — 0/1 Knapsack

State: $dp[i][w]$ = max value using first i items, capacity w . Recurrence:

- Skip item i : $dp[i-1][w]$.
- Take item i (if it fits): $dp[i-1][w - \text{wt}[i]] + \text{val}[i]$.
- $dp[i][w] = \max$ of these.

Space optimization: iterate w from high to low with a 1D array, so each item is used at most once.

```
def knapsack(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for j in range(capacity, w - 1, -1): # reverse!
            dp[j] = max(dp[j], dp[j - w] + v)
    return dp[capacity]
```

Pattern — Unbounded Knapsack (Coin Change)

Same as 0/1, but iterate w forward, so each item can be used multiple times.

Pattern — Longest Common Subsequence (LCS)

State: $dp[i][j]$ = LCS length of $s1[:i]$ and $s2[:j]$. Recurrence:

- If $s1[i-1] == s2[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$ (extend LCS).
- Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ (drop one character).

Pattern — Edit Distance

State: $dp[i][j]$ = min edits to transform $s1[:i]$ into $s2[:j]$. Recurrence:

- If characters match: $dp[i][j] = dp[i-1][j-1]$.
- Else: $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ (delete, insert, substitute).

Pattern — State-machine DP (stock trading)

The trick is identifying the *states* the system can be in. For “Best Time to Buy and Sell Stock with Cooldown”:

- $hold[i]$ = max profit at end of day i , holding a share.
- $sold[i]$ = max profit at end of day i , just sold (must rest tomorrow).
- $rest[i]$ = max profit at end of day i , not holding, not in cooldown.

Transitions:

- $hold[i] = \max(hold[i-1], rest[i-1] - prices[i])$ — keep holding, or buy from rest.
- $sold[i] = hold[i-1] + prices[i]$ — must have held to sell.
- $rest[i] = \max(rest[i-1], sold[i-1])$ — rest, or transition out of cooldown.

Answer: $\max(sold[n-1], rest[n-1])$.

3.5 Interval DP

Problems where you split a range and combine subproblem answers.

State: $dp[i][j]$ = answer on interval $[i, j]$. Recurrence: enumerate a split point k in $[i, j]$:

$dp[i][j] = \min/\max \text{ over } k \text{ of } (dp[i][k] + dp[k+1][j] + \text{cost}(i, k, j))$.

Classic example — Matrix Chain Multiplication: Multiplying matrices of dimensions $(d_0, d_1), (d_1, d_2), \dots, (d_{n-1}, d_n)$. Find the parenthesization minimizing scalar multiplications.

State: $dp[i][j]$ = min cost to multiply matrices i through j . Recurrence: $dp[i][j] = \min \text{ over } k \text{ of } (dp[i][k] + dp[k+1][j] + d_i * d_{k+1} * d_{j+1})$.

Iteration order: by interval length, from smallest to largest.

3.6 DP on Trees

Trees are DAGs, so DP works naturally. Process children before parents (post-order).

House Robber on a Tree (LC 337): Each node has a value. Can't rob both a node and its direct children. Max total loot?

State: for each node, return two values:

- `with_rob`: max loot if we rob this node.
- `without_rob`: max loot if we don't rob this node.

Recurrence:

```
def rob(node):
    if not node: return (0, 0)
    l_with, l_without = rob(node.left)
    r_with, r_without = rob(node.right)
    with_rob = node.val + l_without + r_without
    without_rob = max(l_with, l_without) + max(r_with, r_without)
    return (with_rob, without_rob)
```

3.7 Probability / Expected-Value DP

This is your strong suit given your stochastic modeling background.

Template: state is (current position/configuration). Transitions are probabilistic.

$$E[\text{state}] = \sum P(\text{transition}) \cdot (\text{immediate_reward} + E[\text{next_state}])$$

Often computed backward from absorbing states.

Worked example — Knight Probability in Chessboard (LC 688): Place a knight at (r, c) on an N×N board. It makes k moves, each chosen uniformly among the 8 knight moves (even if off-board — if off-board, game ends). Probability it's still on the board after k moves?

State: $P[k][r][c]$ = probability of being on board at (r, c) after k moves from the start.

Transition: $P[k+1][r'][c'] = \text{sum over } (r, c) \text{ of } P[k][r][c] \cdot (1/8) \text{ for each move } (r, c) \rightarrow (r', c') .$

```
def knight_probability(n, k, r, c):
    moves = [(2,1), (2,-1), (-2,1), (-2,-1), (1,2), (1,-2), (-1,2), (-1,-2)]
    P = [[0]*n for _ in range(n)]
    P[r][c] = 1.0
    for _ in range(k):
        new_P = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if P[i][j] == 0: continue
                for di, dj in moves:
                    ni, nj = i + di, j + dj
                    if 0 <= ni < n and 0 <= nj < n:
                        new_P[ni][nj] += P[i][j] / 8
        P = new_P
    return sum(sum(row) for row in P)
```

3.8 Common DP mistakes

- **Wrong iteration order:** computing $dp[i]$ before its dependencies. Trace through a small case.
- **Off-by-one in indices:** especially with strings, 2D DP, and knapsack. Keep track of whether $dp[i]$ means "using i items" or "ending at index i".
- **Missing base case:** $dp[0]$ is often non-trivial. Think about what it should mean.
- **Confusing "subsequence" and "subarray":** subsequence preserves order but is non-contiguous; subarray is contiguous. They need very different DP.
- **Inefficient state:** if your state has too many dimensions, you'll TLE. Can you drop a dimension? (E.g., stock problems usually don't need $dp[i][\text{transactions}][\text{holding}]$ — just $dp[i][\text{holding}]$ with a small number of states.)

PART 4 — Probability and Math

Given your background, this section focuses less on definitions and more on the computational tricks you'll actually use in problems.

4.1 Core identities to have at your fingertips

Linearity of expectation: $E[X + Y] = E[X] + E[Y]$ — *always*, even if X and Y are dependent.

This is the single most-used identity in probability problems. When a problem asks for an expected count, often the fastest solution is to define an indicator variable for each item and sum the probabilities.

Variance: $\text{Var}(X) = E[X^2] - E[X]^2$ $\text{Var}(aX + b) = a^2\text{Var}(X)$ $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \cdot \text{Cov}(X, Y)$

If X and Y are independent, $\text{Cov}(X, Y) = 0$, so $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.

Covariance: $\text{Cov}(X, Y) = E[XY] - E[X]E[Y]$

Law of total expectation (tower rule): $E[X] = E[E[X | Y]]$

Used constantly in expected-value DP: condition on the next event, take the expectation over its outcomes.

Law of total variance: $\text{Var}(X) = E[\text{Var}(X | Y)] + \text{Var}(E[X | Y])$

Bayes' rule: $P(A | B) = P(B | A) P(A) / P(B)$

4.2 Common distributions — mean, variance, when to use

Distribution	Mean	Variance	Natural use case
Bernoulli(p)	p	p(1-p)	Single success/failure trial
Binomial(n, p)	np	np(1-p)	Number of successes in n trials
Geometric(p)	1/p	(1-p)/p ²	Trials until first success
Poisson(λ)	λ	λ	Count of rare events in fixed interval
Uniform(a, b)	(a+b)/2	(b-a) ² /12	No prior info, fair randomness

Exponential(λ)	$1/\lambda$	$1/\lambda^2$	Time until next event (memoryless)
Normal(μ, σ^2)	μ	σ^2	Sums of many independent effects (CLT)

Memoryless property of Exponential/Geometric: $P(X > s + t \mid X > s) = P(X > t)$. If you've waited s time units without an event, the additional wait time has the same distribution. Useful in queueing and renewal problems.

4.3 Indicator variable technique

Setup: You want $E[\text{count of something}]$. **Trick:** Define $x_i = 1$ if condition i holds, 0 otherwise. Then $\text{count} = \sum x_i$. **By linearity:** $E[\text{count}] = \sum E[x_i] = \sum P(\text{condition } i)$.

Example — expected number of fixed points in a random permutation: Define $x_i = 1$ if position i maps to itself. $P(x_i = 1) = 1/n$ (probability that a specific element lands in a specific spot is $1/n$). There are n positions. So $E[\text{fixed points}] = n \cdot (1/n) = 1$.

Remarkably clean — independence isn't required.

4.4 Reservoir Sampling

Problem: You have a stream of unknown length. Sample k items uniformly at random, using only $O(k)$ space.

Algorithm:

1. Put the first k items in the reservoir.
2. For each item i ($i \geq k$), replace a random reservoir slot with it with probability $k/(i+1)$. Otherwise, skip.

Why it's uniform (proof for $k=1$): After seeing n items, what's the probability item j is in the reservoir?

- Item j is initially placed if $j = 0$: probability 1.
- For each later item $i > j$, item j survives with probability $1 - 1/(i+1) = i/(i+1)$.
- **Product:** $1 \cdot (j+1)/(j+2) \cdot (j+2)/(j+3) \cdot \dots \cdot (n-1)/n = 1/n$.

Each item ends up with probability $1/n$. Uniform sampling, $O(1)$ space.

```
import random
def reservoir_sample(stream, k):
    reservoir = []
```

```

for i, item in enumerate(stream):
    if i < k:
        reservoir.append(item)
    else:
        j = random.randint(0, i)
        if j < k:
            reservoir[j] = item
return reservoir

```

4.5 Fast Exponentiation (Binary Exponentiation)

Problem: Compute a^n in $O(\log n)$ instead of $O(n)$.

Idea: Express n in binary. For each bit, square the running base. Multiply into the answer when the bit is 1.

```

def fast_pow(a, n):
    result = 1
    while n > 0:
        if n & 1:
            result *= a
        a *= a
        n >>= 1
    return result

```

Modular version: just take $\% p$ at every step to keep numbers small.

```

def fast_pow_mod(a, n, p):
    result = 1 % p
    a %= p
    while n > 0:
        if n & 1:
            result = result * a % p
        a = a * a % p
        n >>= 1
    return result

```

Used in cryptography, competitive programming, and anywhere you see "answer modulo 10^9+7 ."

4.6 Modular Arithmetic

Why it matters: Many problems ask for answers modulo a large prime (often 10^9+7) to keep

numbers from exploding.

Basic rules:

- $(a + b) \% p = ((a \% p) + (b \% p)) \% p$
- $(a \cdot b) \% p = ((a \% p) \cdot (b \% p)) \% p$
- **Subtraction requires care:** $(a - b) \% p = ((a - b) \% p + p) \% p$ to avoid negative results.
- **Division:** $a / b \bmod p$ doesn't mean "integer division." You need the *modular inverse* of b.

Modular inverse via Fermat's little theorem: If p is prime and $\gcd(a, p) = 1$, then $a^{(p-1)} \equiv 1 \pmod{p}$, so $a^{(-1)} \equiv a^{(p-2)} \pmod{p}$.

Compute with fast exponentiation.

4.7 Monte Carlo Simulation

When a closed-form solution is hard or impossible, you can estimate probabilities/expectations by simulating many trials.

Law of large numbers: the sample mean converges to the true mean as $n \rightarrow \infty$.

Convergence rate: standard error is $\sigma/\sqrt{n}$. To cut error in half, multiply trials by 4.

Template:

```
import random
def monte_carlo_estimate(trials=100000):
    total = 0
    for _ in range(trials):
        # simulate one experiment, record outcome
        outcome = run_one_trial()
        total += outcome
    return total / trials
```

Use when:

- The state space is too large for DP.
- You need a "sanity check" number before attempting a closed form.
- The problem explicitly asks for an approximate answer within some tolerance.

4.8 Problem archetypes that keep reappearing

“Expected number of attempts until success”: Geometric distribution. $E = 1/p$.

“Expected time to cover all possibilities” (coupon collector): Expected number of random draws (with replacement) to see all n distinct items is $n \cdot (1 + 1/2 + 1/3 + \dots + 1/n) \approx n \cdot \ln(n)$.

“Random walk on a graph”: Often solvable by setting up linear equations for expected hitting times. For a simple path, symmetry or first-step analysis gives closed forms.

“Random permutation with a property”: Indicator variables + linearity of expectation.

“Birthday paradox”: $P(\text{at least one collision among } k \text{ people in a range of } n) \approx 1 - \exp(-k^2/(2n))$. Useful in hashing analysis.

“Drawing without replacement”: Hypergeometric distribution, but often approached by symmetry.

Closing note

This covers the conceptual depth you need for parts 1–4 of the prep material. The common thread across all four areas is *choose the right abstraction* — the right data structure, the right algorithm, the right recurrence, the right probability technique. Most mistakes in timed assessments aren't from not knowing any of these; they're from picking the wrong one or from clumsy implementation of the right one.

Practice until the choice becomes automatic. On test day, read the problem twice, name the pattern, verify the complexity will fit, and only then start typing.

加油!